

Video Transcript

Video 9: Cassandra: A Distributed Scalable Database (09:43)

A relational database management system gives you a tremendous amount of functionality, but it also gives you a tremendous amount of structure. Document databases, on the other hand, give you a lot of flexibility, and as you grow up, you might need some of that structure. On the other hand, when it comes to key value data stores, we have tremendous speed in many of the implementations. Now what about scalability and being able to scale with ease? We can make the comparison here to relational database management systems that do pretty well at low scale. But as you scale, there are some scalability challenges that certainly can be done transparently, that is without you really thinking about it. Now when it comes to that last need, the need to be able to scale with ease, Cassandra was set up with that in mind.

So, it is not going for the features of the relational database management systems, nor the speed or flexibility, but for scalability with ease and it makes sense that it was introduced initially by Facebook and that they learned from the lessons of the first implementations that really went for that large scale, which were databases such as Amazon's Dynamo or Google's Big table, where they were looking at reliable performance data and in other cases, just rich data models that were able to grow without a lot of user intervention.

And so in this case, as you can see here, simply put, the goal is for organizations to use it to handle large data, volumes of data in a distributed fashion, and you can see here some of those points being emphasized that you have a master list model that is decentralized. So, you can tolerate failures even up to a datacenter if you have multiple others around the world. If you have, if you have the need to be able to scale, you can simply address it with hardware and when it comes to the hardware, it doesn't have to be specialized. It's commodity servers and that's a great thing to have, simply adding servers instead of people. People are more expensive than machines now and you have a master list replication. You don't have to worry about being having to maintain and scale that master on it's own. And so you can get a sense of the scalability that we're talking about here is a benchmark made by Netflix and you can see here that they're writing at the rate of over a million writes per second on 288 nodes across several clusters. I've provided the link there at the bottom of the slide in case you want to read more about it. So,

with that in mind, as before, we're going to go ahead and create a container of this large scalable database, but we're going to do so local in the same way we have done before for others.

So, as you can see, here is the command to be able to do so. You can see Docker run, the name of the image is Cassandra, the port is 9042, and the container name is Cassandra and we're going to run that detached. So, let's go ahead and paste that command onto our terminal and let's go ahead and run it and as you can see there it is pulling that image, creating a local instance and I'm going to go ahead and open up the dashboard here so we can see when that image comes, it's instantiated and then is ready to go and as you can see there, we now have the green color of our container, meaning that that container is up and running and it's healthy. So, now let's go ahead and close our dashboard and let's go ahead and exit from our terminal and let's go ahead and open up Visual Studio. Into it I'm hardware and to go ahead and drag and drop a folder that I have with a couple of files, as we have done before. One of them will be to write to Cassandra, the other one will be to read from it. So, let's go ahead and get started with the write. We're going to start off by installing the container.

So, we're going to go ahead and copy and paste onto our command line here and as you can see there in my local environment, I have already installed that. So, I get simply the confirmation. I'm going to go ahead and close the terminal and we're going to continue with our code here. You can see there that I am importing the driver and then making a reference to datetime as we have done previously as well and now I'm going to make a connection here to the database. And note that I'm using a term here called keyspace, you can think of that as the database in the context of Cassandra. I'm then going to create something that looks like SQL and it's pretty close to SQL as you can see there and I am creating a key space if no, if it does not exist, and I'm going to call it Pluto and here this is the equivalent of being able to create a database, if that database does not exist.

We're then going to set for the session the key space of Pluto, and that would be the equivalent on relational traditional SQL of saying use in the name of the database. We're then going to create a table and note that we're executing into the database, into the session on each of those blocks. In this case, I'm simply going to create a table and then once I do that, I'm going to go ahead and write into that table and that table is going to have two columns, as you can see there, and the first one is simply going to be a dummy string that I'm going to be writing in and then I'm going to have the timestamp.

So, let's go ahead and save this and now I'm going to go ahead and open up my console. Once again, let's go ahead and clear. Let's go ahead and run that program, Python 3, and then I'm going to use the write file and as you can see there, that was written into the Cassandra database. So, now let's go ahead and read from that database. I'm going to go ahead and open up the read file. We'll start off here with our driver. We'll then make our connection to the database and set Pluto as the key space, the database that we're going to be using. Then we're going to read all of the rows of the table post, and then we will write those to the console. So, I'm going to go ahead and save the file, open up my console, and then I'm going to go ahead and run, in this case, the read file and as you can see there, we have read successfully that row back the one that we wrote before. Let's go ahead and write three times to that database and then let's go ahead and read it and as you can see there, we get the timestamps that we put in back as expected. So, now it's your turn. Go ahead and walk through the code. Make sure you can replicate what we have done, that you can create your containers, that you can run the code. In addition to that, we want you to go ahead and explore a different type of client. In the past we have made recommendation for visual managers, that is a visual graphical user interface that you can use to connect to the database and see the components in a visual way. In this case, we will ask you to log into the container and start up a shell and use what is called the cqlsh and in this case that's a cryptic way of saying the SQL interface onto this database.

So, let me go ahead and show you how that is done. Using the container dashboard, you'll see that there is an icon that allows you to do that, and this is a graphical way to do it. You can also just do everything purely through the command line. But as you can see here, there are a few options, and the second one is to open up the CLI and as you can see there, that launches a new window and you're now inside of the container at the command prompt. Now if we go ahead and enter that CQL shell and hit return, you can see there that we're now at the prompt for Cassandra. Now we can go ahead and say use Pluto and as you can see there when I forgot the semicolon to end my command there, you can see there that you now are using Pluto. At this point we could enter, select star from post, and once again, I did not enter the semicolon and you can see there that we would get back the timestamps that we did previously by running a Python program. So, once again, it is now your turn. Go ahead and dig in, try the code, build the containers, log into the containers and run your shell commands here at the command prompt.